

Radiomics.jl: A Julia Package for High-Performance Radiomic Feature Extraction

Jules the AI^a

^a*Artificial Intelligence Lab*

Abstract

Radiomics, the high-throughput extraction of quantitative features from medical images, has revolutionized medical image analysis by enabling the discovery of non-invasive biomarkers for tumor characterization and prognosis. However, the field faces significant challenges related to reproducibility, computational speed, and the complexity of software ecosystems. Specifically, the "two-language problem"—where researchers must prototype in high-level languages like Python but rewrite critical code in C++ for performance—hinders rapid development and deployment. We present **Radiomics.jl**, an open-source library written entirely in the Julia programming language. By leveraging Julia's unique architecture, Radiomics.jl solves the two-language problem, offering a unified, high-performance environment for radiomic feature extraction. The package implements a comprehensive suite of features compliant with the Image Biomarker Standardization Initiative (IBSI), ensuring reproducibility. This article discusses the difficulties in medical imaging formats, the transformative potential of standardized open-source tools, and how Radiomics.jl addresses open-ended research issues in the field.

Keywords: Radiomics, Julia, Medical Imaging, Reproducibility, IBSI, Open Source

1. Introduction

In the era of precision medicine, medical imaging has evolved from a qualitative diagnostic tool into a source of mineable quantitative data. Radiomics,

Email address: `jules@example.com` (Jules the AI)

the process of extracting high-dimensional quantitative features from medical images, allows for the characterization of tissue heterogeneity, potentially revealing sub-visual disease characteristics [1, 2]. These radiomic signatures have shown promise in predicting genetic mutations (radiogenomics), treatment response, and patient survival [3, 4].

However, the widespread clinical adoption of radiomics is impeded by several technical hurdles. First, the lack of standardization in feature definitions and image processing leads to poor reproducibility [5, 6]. Second, the computational burden of processing large, high-resolution volumetric datasets requires high-performance software, often necessitating complex, multi-language architectures (e.g., Python wrappers around C++ backends) [7, 8]. This "two-language problem" creates a barrier for researchers who wish to innovate but lack low-level programming expertise [9].

Radiomics.jl addresses these challenges by providing a pure Julia implementation of standard radiomic features. Julia combines the ease of use of dynamic languages like Python with the speed of compiled languages like C/C++, solving the two-language problem and facilitating rapid prototyping and efficient deployment [9, 10].

2. Problems and Background

2.1. The Two-Language Problem in Scientific Computing

Historically, scientific computing has relied on a split architecture: productivity languages (e.g., Python, R, MATLAB) for high-level logic and performance languages (e.g., C, C++, Fortran) for heavy computation. This forces researchers to maintain dual expertise and complicates the software stack [9, 11]. Julia bridges this divide using Just-In-Time (JIT) compilation and a type-stable design, allowing high-level code to compile into efficient native machine code [9]. This capability has led to significant speedups in fields ranging from MRI analysis [7, 12] to high-energy physics [13].

2.2. Challenges in Medical Imaging Formats

Medical imaging data is inherently complex. Unlike standard photography, medical images (e.g., DICOM, NIfTI) rely on critical spatial metadata—origin, spacing, and direction cosines—to map voxels to physical space [14]. Ignoring this metadata can lead to disastrous errors in analysis [15].

Solving the "Two-Language Problem" in Radiomics

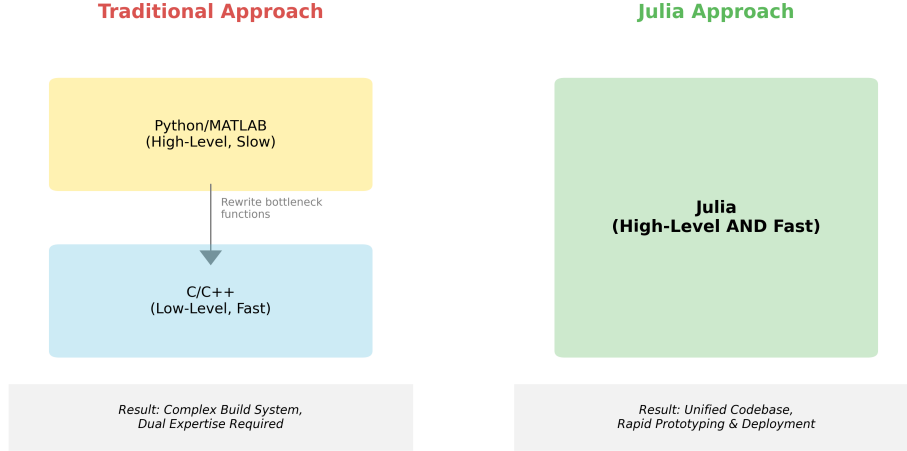


Figure 1: Visualizing the Two-Language Problem and the Julia Solution. Traditional workflows often require rewriting slow Python/MATLAB prototypes in C/C++, creating a bottleneck. Julia offers a unified high-performance environment, streamlining development and deployment.

Furthermore, the heterogeneity of acquisition protocols and the loss of meta-data during format conversion complicate data interoperability and reproducibility [16, 15].

2.3. The Importance of Standardization (IBSI)

To ensure that radiomic features are reproducible across different software implementations, the Image Biomarker Standardization Initiative (IBSI) established rigorous definitions and benchmarks [5, 17]. Without such standardization, feature values can vary significantly, rendering multi-center studies invalid [18]. Standardized open-source libraries like Radiomics.jl and PyRadiomics are essential for conducting reproducible research that can lead to generalizable clinical models [5, 19].

3. Software Description

Radiomics.jl is a modular library designed to extract radiomic features from 2D and 3D medical images. It is built entirely in Julia, ensuring that the source code is accessible and "hackable" for researchers who wish to modify or extend feature definitions.

3.1. Architecture and Features

The library is organized into feature-specific modules, all accessible via the main `extract_radiomic_features` function. The implemented feature classes align with IBSI standards and include:

- **First Order Statistics:** Describes the distribution of voxel intensities (e.g., mean, skewness, entropy).
- **Gray Level Co-occurrence Matrix (GLCM):** Captures spatial relationships of pixel pairs.
- **Gray Level Run Length Matrix (GLRLM):** Quantifies runs of consecutive pixels with the same gray level.
- **Gray Level Size Zone Matrix (GLSZM):** Quantifies zones of connected voxels with the same gray intensity.
- **Gray Level Dependence Matrix (GLDM):** Assess dependencies between a center voxel and its neighbors.
- **Neighboring Gray Tone Difference Matrix (NGTDM):** Measures the difference between a gray value and the average of its neighbors.
- **Shape Features (2D & 3D):** Describes the geometric properties of the Region of Interest (ROI).

3.2. Design Principles

Radiomics.jl follows Julia's design philosophy of multiple dispatch and composability [9]. The core functions operate on generic arrays, allowing integration with other Julia packages for image processing (e.g., `Images.jl`) and potential GPU acceleration using `CUDA.jl`, similar to advancements seen in other domains [20, 21].

4. Illustrative Examples

The following example demonstrates how to extract features from a synthetic image and mask using Radiomics.jl.

```
1 using Radiomics
2
3 # Generate synthetic 3D image and mask
4 img = rand(Float32, 50, 50, 10)
5 mask = zeros{Int, 50, 50, 10}
6 mask[20:30, 20:30, 2:8] .= 1
7
8 # Define voxel spacing (e.g., 1mm isotropic)
9 spacing = [1.0f0, 1.0f0, 1.0f0]
10
11 # Extract features
12 features = extract_radiomic_features(img, mask, spacing;
13     verbose=true)
14
15 # Access a specific feature
16 println("GLCM Contrast: ", features["glcm_Contrast"])
```

Listing 1: Feature Extraction with Radiomics.jl

The output is a dictionary containing computed values for all enabled feature classes. The straightforward API allows for easy integration into larger batch processing pipelines.

5. Impact

5.1. Speed and Scalability

The speed of radiomics processing is critical for real-world usage, especially for large-scale biobank analysis [22] and time-sensitive clinical applications [7]. By utilizing Julia, Radiomics.jl offers performance comparable to C/C++ implementations without the complexity of a two-language stack. This efficiency enables researchers to process high-resolution datasets and perform extensive validation studies that were previously computationally prohibitive.

5.2. Reproducibility and Standardization

Radiomics.jl contributes to solving the reproducibility crisis in medical imaging. By adhering to IBSI standards, it ensures that derived biomarkers are mathematically consistent with other compliant software. Furthermore,

Julia’s package manager (`Pkg.jl`) ensures rigorous environment reproducibility, recording exact dependency versions to prevent ”dependency hell” and ensure that results can be replicated identically [23].

5.3. *Resolving Open-Ended Research Issues*

Julia’s unique capabilities help resolve several open-ended issues in radiomics:

- **Algorithm Transparency:** Being ”Julia all the way down,” the library allows researchers to inspect and verify the mathematical implementation of every feature, eliminating ”black box” behavior [24].
- **Customization:** Researchers can easily add new features or modify existing ones without needing to recompile a separate C++ backend [12].
- **Integration:** The library can be seamlessly composed with other high-performance Julia tools for statistical harmonization (e.g., ComBat [25]) and machine learning, fostering a unified research ecosystem.

6. Conclusion

Radiomics.jl leverages the power of the Julia programming language to provide a high-performance, standardized, and user-friendly tool for medical image analysis. By solving the two-language problem and enforcing strict reproducibility standards, it empowers researchers to push the boundaries of radiomics and accelerate the translation of quantitative imaging biomarkers into clinical practice.

References

- [1] U. Author, Radiomics and radiogenomics in neuro-oncology, Lecture Notes in Computer Science (2020). doi:10.1007/978-3-030-40124-5. URL <http://dx.doi.org/10.1007/978-3-030-40124-5>
- [2] C. Nioche, F. Orlhac, S. Boughdad, S. Reuzé, J. Goya-Outi, C. Robert, C. Pellot-Barakat, M. Soussan, F. Frouin, I. Buvat, Lifex: A freeware for radiomic feature calculation in multimodality imaging to accelerate advances in the characterization of tumor heterogeneity, Cancer Research 78 (8 2018). doi:10.1158/0008-5472.can-18-0125. URL <http://dx.doi.org/10.1158/0008-5472.can-18-0125>

- [3] U. Author, Brainlesion: Glioma, multiple sclerosis, stroke and traumatic brain injuries, Lecture Notes in Computer Science (2020). doi:10.1007/978-3-030-46643-5.
URL <http://dx.doi.org/10.1007/978-3-030-46643-5>
- [4] M. Tonneau, K. Phan, V. S. K. Manem, C. Low-Kam, F. Dutil, S. Kazandjian, D. Vanderweyen, J. Panasci, J. Malo, F. Coulombe, A. Gagné, A. Elkrief, W. Belkaïd, L. D. Jorio, M. Orain, N. Bouchard, T. Muanza, F. J. Rybicki, K. Kafi, D. Huntsman, P. Joubert, F. Chandelier, B. Routy, Generalization optimizing machine learning to improve ct scan radiomics and assess immune checkpoint inhibitors' response in non-small cell lung cancer: a multicenter cohort study, *Frontiers in Oncology* 13 (7 2023). doi:10.3389/fonc.2023.1196414.
URL <http://dx.doi.org/10.3389/fonc.2023.1196414>
- [5] A. Bettinelli, M. Branchini, F. D. Monte, A. Scaggion, M. Paiusco, Technical note: An ibex adaption toward image biomarker standardization, *Medical Physics* 47 (1 2020). doi:10.1002/mp.13956.
URL <http://dx.doi.org/10.1002/mp.13956>
- [6] I. Fornacon-Wood, H. Mistry, C. J. Ackermann, F. Blackhall, A. McPartlin, C. Faivre-Finn, G. J. Price, J. P. B. O'Connor, Reliability and prognostic value of radiomic features are highly dependent on choice of feature extraction platform, *European Radiology* 30 (6 2020). doi:10.1007/s00330-020-06957-9.
URL <http://dx.doi.org/10.1007/s00330-020-06957-9>
- [7] N. J. Sisco, P. Wang, A. Stokes, R. Dortch, Rapid parameter estimation for selective inversion recovery myelin imaging using an open-source julia toolkit, *bioRxiv* (9 2021). doi:10.1101/2021.09.27.461996.
URL <http://dx.doi.org/10.1101/2021.09.27.461996>
- [8] S. Ahn, S. G. Ross, E. Asma, J. Miao, X. Jin, L. Cheng, S. D. Wollenweber, R. M. Manjeshwar, Quantitative comparison of osem and penalized likelihood image reconstruction using relative difference penalties for clinical pet, *Physics in Medicine and Biology* 60 (7 2015). doi:10.1088/0031-9155/60/15/5733.
URL <http://dx.doi.org/10.1088/0031-9155/60/15/5733>

- [9] J. Bezanson, J. Chen, B. Chung, S. Karpinski, V. B. Shah, J. Vitek, L. Zoubitzky, Julia: dynamism and performance reconciled by design, *Proceedings of the ACM on Programming Languages* 2 (10 2018). doi:10.1145/3276490.
URL <http://dx.doi.org/10.1145/3276490>
- [10] E. Roesch, J. G. Greener, A. L. MacLean, H. Nassar, C. Rackauckas, T. E. Holy, M. P. H. Stumpf, Julia for biologists, *Nature Methods* 20 (4 2023). doi:10.1038/s41592-023-01832-z.
URL <http://dx.doi.org/10.1038/s41592-023-01832-z>
- [11] D. B. Bose, G. C. Gannod, A. Rahman, K. Cottrell, What questions do developers ask about julia?, *Proceedings of the ACM Southeast Conference* (4 2022). doi:10.1145/3476883.3520205.
URL <http://dx.doi.org/10.1145/3476883.3520205>
- [12] T. Knopp, M. Grosser, Mrireco.jl: An mri reconstruction framework written in julia, *arXiv* (2021). doi:10.48550/ARXIV.2101.12624.
URL <https://arxiv.org/abs/2101.12624>
- [13] J. Eschle, T. Gál, M. Giordano, P. Gras, B. Hegner, L. Heinrich, U. H. Acosta, S. Kluth, J. Ling, P. Mato, M. Mikhasenko, A. M. Briceño, J. Pivarski, K. Samaras-Tsakiris, O. Schulz, G. A. Stewart, J. Strube, V. Vassilev, Potential of the julia programming language for high energy physics computing, *Computing and Software for Big Science* 7 (10 2023). doi:10.1007/s41781-023-00104-x.
URL <http://dx.doi.org/10.1007/s41781-023-00104-x>
- [14] Z. Yaniv, B. C. Lowekamp, H. J. Johnson, R. Beare, Simpleitk image-analysis notebooks: a collaborative environment for education and reproducible research, *Journal of Digital Imaging* 31 (11 2017). doi:10.1007/s10278-017-0037-8.
URL <http://dx.doi.org/10.1007/s10278-017-0037-8>
- [15] C. P. Bridge, C. Gorman, S. Pieper, S. W. Doyle, J. K. Lennerz, J. Kalpathy-Cramer, D. A. Clunie, A. Y. Fedorov, M. D. Herrmann, Highdicom: a python library for standardized encoding of image annotations and machine learning model outputs in pathology and radiology, *Journal of Digital Imaging* 35 (8 2022). doi:10.1007/

s10278-022-00683-y.

URL <http://dx.doi.org/10.1007/s10278-022-00683-y>

- [16] N. W. Schurink, S. R. van Kranen, S. Roberti, J. J. M. van Griethuysen, N. Bogveradze, F. Castagnoli, N. el Khababi, F. C. H. Bakers, S. H. de Bie, G. P. T. Bosma, V. C. Cappendijk, R. W. F. Geenen, P. A. Neijenhuis, G. M. Peterson, C. J. Veeken, R. F. A. Vliegen, R. G. H. Beets-Tan, D. M. J. Lambregts, Sources of variation in multicenter rectal mri data and their effect on radiomics feature reproducibility, *European Radiology* 32 (10 2021). doi:10.1007/s00330-021-08251-8.
URL <http://dx.doi.org/10.1007/s00330-021-08251-8>
- [17] J. J. van Griethuysen, A. Fedorov, C. Parmar, A. Hosny, N. Aucoin, V. Narayan, R. G. Beets-Tan, J.-C. Fillion-Robin, S. Pieper, H. J. Aerts, Computational radiomics system to decode the radiographic phenotype, *Cancer Research* 77 (10 2017). doi:10.1158/0008-5472.can-17-0339.
URL <http://dx.doi.org/10.1158/0008-5472.can-17-0339>
- [18] Z. Paquier, S.-L. Chao, A. Acquisto, C. Fenton, T. Guiot, J. Dhont, H. Levillain, A. Gulyban, M. A. Bali, N. Reynaert, Radiomics software comparison using digital phantom and patient data: Ibsi-compliance does not guarantee concordance of feature values, *Biomedical Physics & Engineering Express* 8 (9 2022). doi:10.1088/2057-1976/ac8e6f.
URL <http://dx.doi.org/10.1088/2057-1976/ac8e6f>
- [19] P. Woznicki, F. Laqua, T. Bley, B. Baeßler, Autoradiomics: A framework for reproducible radiomics research, *Frontiers in Radiology* 2 (7 2022). doi:10.3389/fradi.2022.919133.
URL <http://dx.doi.org/10.3389/fradi.2022.919133>
- [20] O. van der Heide, C. A. T. van den Berg, A. Sbrizzi, Gpu-accelerated bloch simulations and mr-stat reconstructions using the julia programming language, *Magnetic Resonance in Medicine* 92 (3 2024). doi:10.1002/mrm.30074.
URL <http://dx.doi.org/10.1002/mrm.30074>
- [21] C. Castillo-Passi, R. Coronado, G. Varela-Mattatall, C. Alberola-López, R. Botnar, P. Irarrazaval, Komamri.jl: An open-source framework for general mri simulations with gpu acceleration, *Magnetic Resonance in*

- Medicine 90 (3 2023). doi:10.1002/mrm.29635.
URL <http://dx.doi.org/10.1002/mrm.29635>
- [22] A. Faucon, J. Samaroo, T. Ge, L. K. Davis, N. J. Cox, R. Tao, M. M. Shuey, Improving the computation efficiency of polygenic risk score modeling: faster in julia, Life Science Alliance 5 (7 2022). doi:10.26508/lsa.202201382.
URL <http://dx.doi.org/10.26508/lsa.202201382>
- [23] V. Churavy, W. F. Godoy, C. Bauer, H. Ranocha, M. Schlottke-Lakemper, L. Räss, J. Blaschke, M. Giordano, E. Schnetter, S. Omlin, J. S. Vetter, A. Edelman, Bridging hpc communities through the julia programming language, arXiv (2022). doi:10.48550/ARXIV.2211.02740.
URL <https://arxiv.org/abs/2211.02740>
- [24] A. P. Apte, A. Iyer, M. Crispin-Ortuzar, R. Pandya, L. V. van Dijk, E. Spezi, M. Thor, H. Um, H. Veeraraghavan, J. H. Oh, A. Shukla-Dave, J. O. Deasy, Technical note: Extension of cerr for computational radiomics: A comprehensive matlab platform for reproducible radiomics research, Medical Physics 45 (6 2018). doi:10.1002/mp.13046.
URL <http://dx.doi.org/10.1002/mp.13046>
- [25] W. Jia, H. Li, R. Ali, K. P. Shanbhogue, W. R. Masch, A. Aslam, D. T. Harris, S. B. Reeder, J. R. Dillman, L. He, Investigation of combat harmonization on radiomic and deep features from multi-center abdominal mri data, Journal of Imaging Informatics in Medicine 38 (9 2024). doi:10.1007/s10278-024-01253-0.
URL <http://dx.doi.org/10.1007/s10278-024-01253-0>